

System Architecture Document

AI Resume Screening & Candidate Ranking System

Project: LaunchED | AI Resume Screening Capstone

Team: SomethingCo

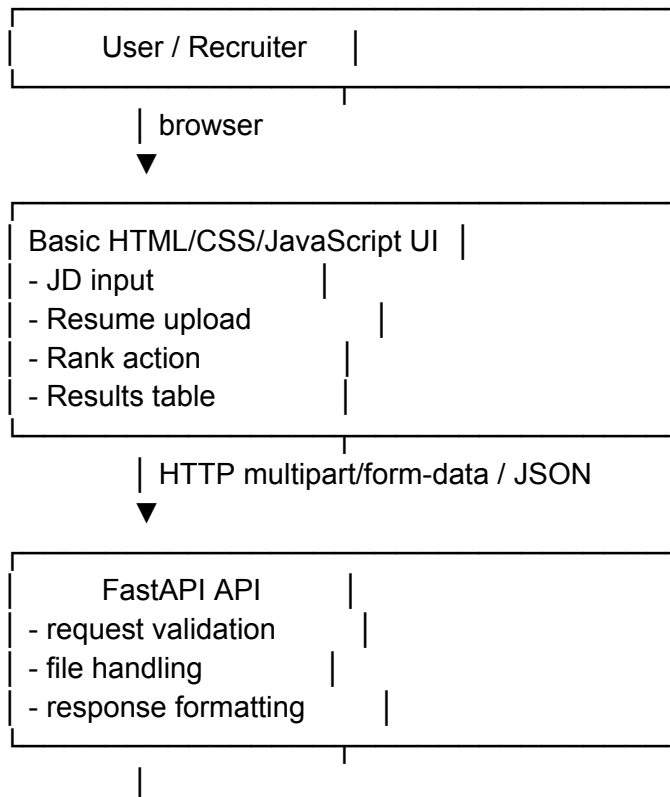
Deadline: 31 August 2026

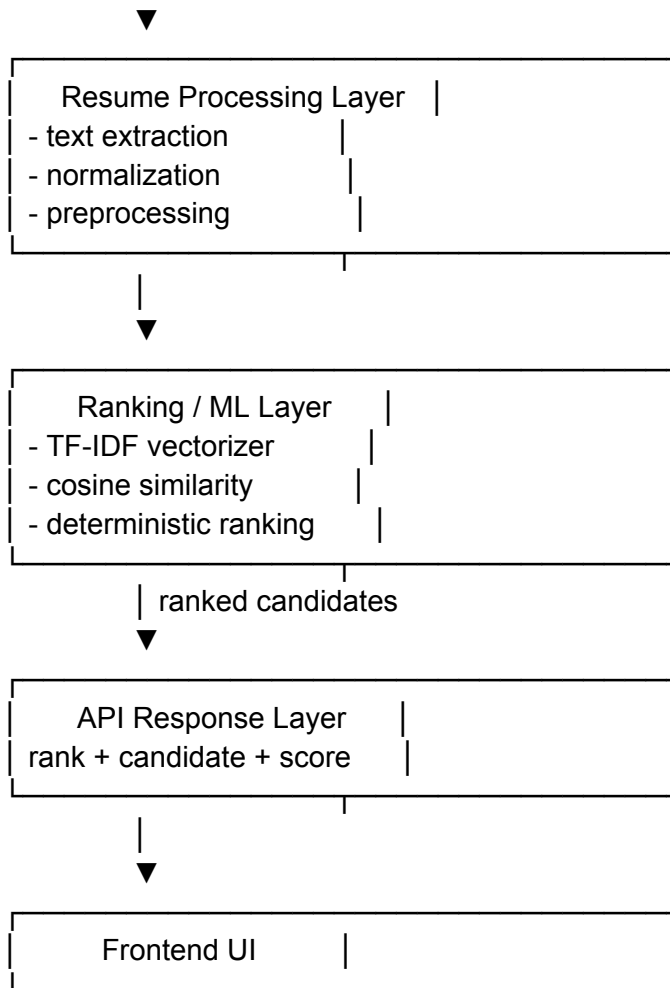
1. Architecture goal

Provide a simple, reliable end-to-end application in which a user submits one job description and multiple resumes, the backend executes the NLP ranking pipeline, and the frontend displays ranked candidates.

The architecture deliberately minimizes integration risk because the Linear plan treats today as the build deadline.

2. High-level architecture





3. Components

3.1 Frontend

A minimal browser interface is sufficient for the prescribed requirement.

Responsibilities:

- accept job description;
- select multiple supported resumes;
- send request to backend;
- show loading/error state;
- render ranked candidates and similarity scores.

Implementation decision: plain HTML/CSS/JavaScript to avoid framework setup overhead.

3.2 FastAPI backend

Responsibilities:

- expose the ranking endpoint;
- validate inputs;
- receive uploaded files;
- call the processing/ranking services;
- return structured results;
- return useful error messages.

Suggested interface:

POST /rank

Input:

- job description;
- multiple resume files.

Output:

```
{
  "candidates": [
    {"rank": 1, "name": "Candidate A", "score": 0.874},
    {"rank": 2, "name": "Candidate B", "score": 0.742}
  ]
}
```

The exact response schema may be adjusted during implementation, but the contract should remain stable between frontend and backend.

3.3 Processing layer

Responsibilities:

- identify supported file types;
- extract text;
- normalize and preprocess text;
- return clean text to the ranking layer.

This layer should not contain UI logic or ranking-specific presentation logic.

3.4 Ranking layer

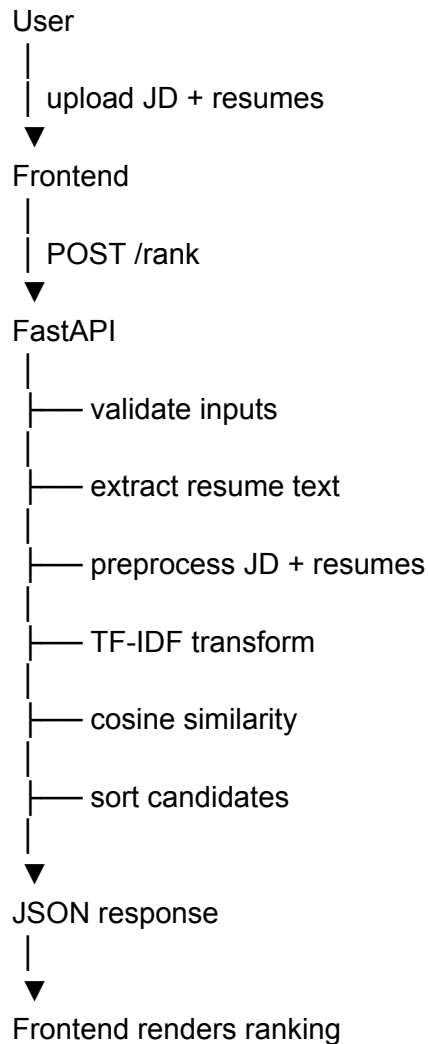
Responsibilities:

- construct the common TF-IDF feature space;

- transform JD and resumes;
- calculate cosine similarity;
- sort candidates deterministically;
- return scores and ranks.

The same implementation should be callable from both the notebook and API where practical, reducing duplicated logic.

4. End-to-end request sequence



5. Deployment model

For the capstone deadline, the primary deployment target is **local execution** unless an external deployment is later required or already available. This reduces infrastructure risk while still providing a demonstrable application.

Expected local flow:

- start FastAPI server;
- open frontend in browser;
- submit sample JD/resumes;
- observe ranking.

Cloud deployment is explicitly out of the critical path.

6. Storage architecture

A database is not required by the Option 2 brief or current Linear plan. Therefore the MVP should be stateless:

- uploaded files are processed for the request;
- ranking results are returned to the user;
- persistent candidate storage is not required.

If temporary files are created, they should be deleted after processing where practical.

7. Security and privacy

Because resumes contain personal information, the implementation should:

- use only authorized/sample data for the capstone demo;
- avoid logging full resume contents;
- avoid persistent storage unless explicitly required;
- validate uploaded file types and sizes;
- never present the ranking as an autonomous employment decision.

8. Reliability and failure handling

Input failures

- unsupported file → clear validation error;
- empty/failed extraction → candidate flagged or request rejected;
- missing JD → validation error;
- no resumes → validation error.

ML failures

- no vocabulary overlap → return a low/zero similarity score rather than crashing;
- malformed text → normalize or flag the input.

API failures

- return structured HTTP errors;
- frontend displays actionable error text.

9. Observability for the capstone

Full production observability is unnecessary. The project should retain enough evidence to debug and demonstrate the system:

- request success/failure;
- number of resumes processed;
- ranking output;
- evaluation results in the notebook.

Do not log sensitive resume text by default.

10. Architecture boundaries

Frontend owns: user interaction and presentation.

API owns: transport, validation, orchestration.

Processing owns: document-to-text conversion and preprocessing.

ML layer owns: vectorization, similarity, ranking, evaluation logic.

Notebook owns: reproducible analysis and evidence.

11. Architectural decisions

Decision	Rationale
FastAPI	Meets backend requirement with low implementation overhead
HTML/CSS/JS	Meets basic frontend requirement without framework overhead
TF-IDF + cosine	Explicitly prescribed by Option 2

Stateless MVP Database is not required; reduces scope and privacy risk

Local-first Maximizes probability of a working demo before deadline

Shared ranking
module Prevents notebook/API logic drift

12. Definition of done

The architecture is implemented when a user can complete:

open UI → enter JD → upload multiple resumes → submit → API processes
→ ranked candidates displayed

and the same core ranking logic is demonstrated in the notebook with evaluation evidence.